



Mutation testing with StrykerJS

– LABORATORY –



Introduction

In this laboratory, you will carry out some practical exercises with a state-of-the-art Mutation testing tool: StrykerJS. StrykerJS is the industry leading mutation testing tool for JavaScript and TypeScript, developed in open source and maintained by Info Support.

The code base you will be working on is [numbers.js](#), a JavaScript library to perform math operations, it has simple functions like `sum` and `max`, but also an implementation for complex numbers, matrix calculus, etc. Running StrykerJS on the 'basic.js' file yields a mutation score of 76.29%.

To become familiar with StrykerJS, you will first do an introductory exercise where we improve the mutation score. This will give you some hands-on experience with unit testing in JavaScript. Afterwards, you will be challenged to enhance StrykerJS with your very own mutator. Good luck!



1. Numbers.js

1.1. Setup and run StrykerJS

During this exercise you will play around with the code base. You will learn the basics of compiling StrykerJS and running it on the `basic.js` file in `numbers.js`.

To get started, be sure you have [vscode](#) and [an LTS version of nodejs installed](#).

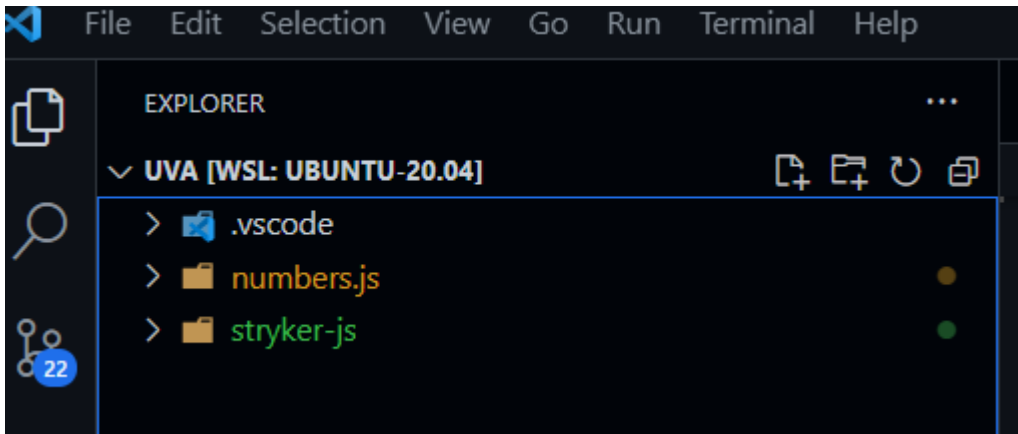
```
$ node -v  
v24.18.0
```

You will also need to install `pnpm`, the package manager used by StrykerJS.

```
$ npx get-pnpm  
# source ~/.bashrc or ~/.zshrc depending on your shell, or restart your shell /  
command prompt.  
$ pnpm -v  
# 12.3.4
```

Also make sure to have `bash` (linux shell) or `git-bash` (windows) installed.

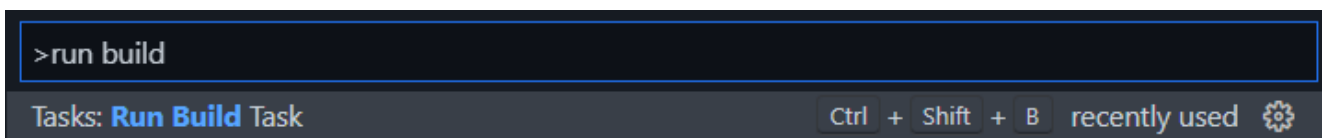
Next, unzip the "assignment.zip" file on your hard disk and open the assignment.code-workspace file from the labs/assignment directory. THIS IS IMPORTANT, the workspace is configured so the nested numbers.js and stryker-js folders are picked up correctly.



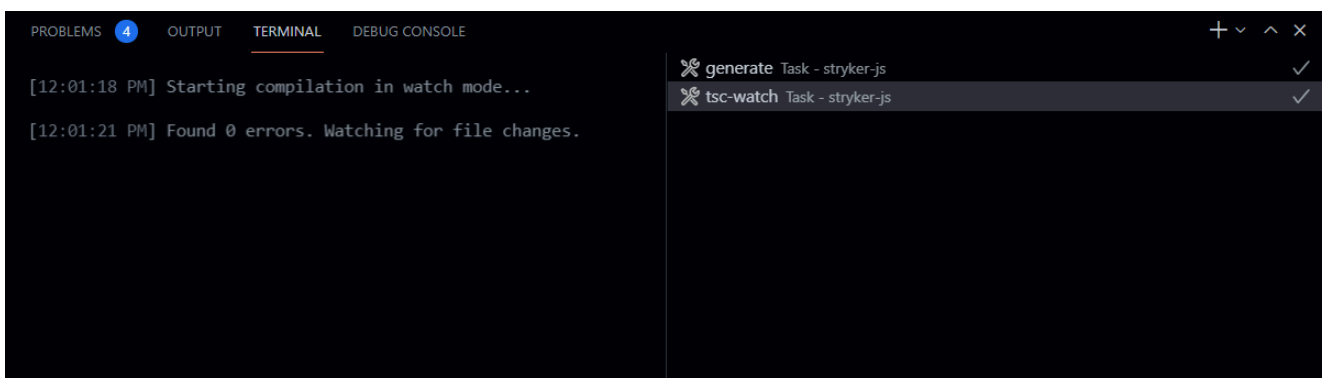
Now to install the dependencies, cd into "numbers.js" and perform `npm install`. After that, run `pnpm install` in "stryker-js".

```
$ cd numbers.js
$ npm install
$ cd ../stryker-js
$ pnpm install
```

You should be able to start the StrykerJS TypeScript compiler in watch mode by pressing CTRL+shift+B (or CMD+shift+B on Mac). Alternatively, you can use F1 and select "Tasks: Run Build Task".



Now wait for StrykerJS to be compiled for the first time.



The build command task will remain running after the first compilation. Any changes you make will be recompiled on the fly.

After StrykerJS is compiled, it is time for your first StrykerJS run. You can do this by executing the "run-stryker.sh" script in the "numbers.js" directory.

```
$ cd ../numbers.js
$ ./run-stryker.sh
```

Running Stryker this way might take a few minutes:

```
nicojs@NicoJ01 ~/uva/2023/numbers.js $ ./run-stryker.sh
07:49:13 (6606) INFO ProjectReader Found 1 of 37 file(s) to be mutated.
07:49:13 (6606) INFO Instrumenter Instrumented 1 source file(s) with 506 mutant(s)
07:49:13 (6606) INFO ConcurrencyTokenProvider Creating 4 test runner process(es).
07:49:14 (6606) INFO DryRunExecutor Starting initial test run (mocha test runner with "perTest" coverage analysis). This may take a while.
07:49:21 (6606) INFO DryRunExecutor Initial test run succeeded. Ran 136 tests in 7 seconds (net 6795 ms, overhead 385 ms).
Mutation testing [=====] 100% (elapsed: <1m, remaining: n/a) 429/429 Mutants tested (77 survived, 12 timed out)
```

NOTE: The next time you run StrykerJS, it will be much faster because we've got incremental enabled in the config file.

When Stryker is done, you can open up the HTML report in your browser. The file is located here: "numbers.js/reports/mutation/mutation.html".

All files

Mutants Tests

All files												
1549 376 154												
File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
All files	74.51	80.47	1435	376	114	154	445	3	0	1549	530	2527
basic.js	73.90	80.81	309	76	11	37	77	0	0	320	113	510
calculus.js	42.62	57.14	47	39	5	31	28	0	0	52	70	150
complex.js	96.55	96.55	55	2	1	0	16	0	0	56	2	74
dsp.js	84.38	87.10	24	4	3	1	8	0	0	27	5	40
generators.js	93.62	93.62	34	3	10	0	13	0	0	44	3	60
matrix.js	72.90	78.88	653	188	49	73	222	0	0	702	261	1185
prime.js	82.89	85.64	134	26	21	6	24	0	0	155	32	211
random.js	75.21	75.83	80	29	11	1	18	3	0	91	30	142
statistic.js	87.93	91.89	99	9	3	5	39	0	0	102	14	155

1.2. Exploring the environment

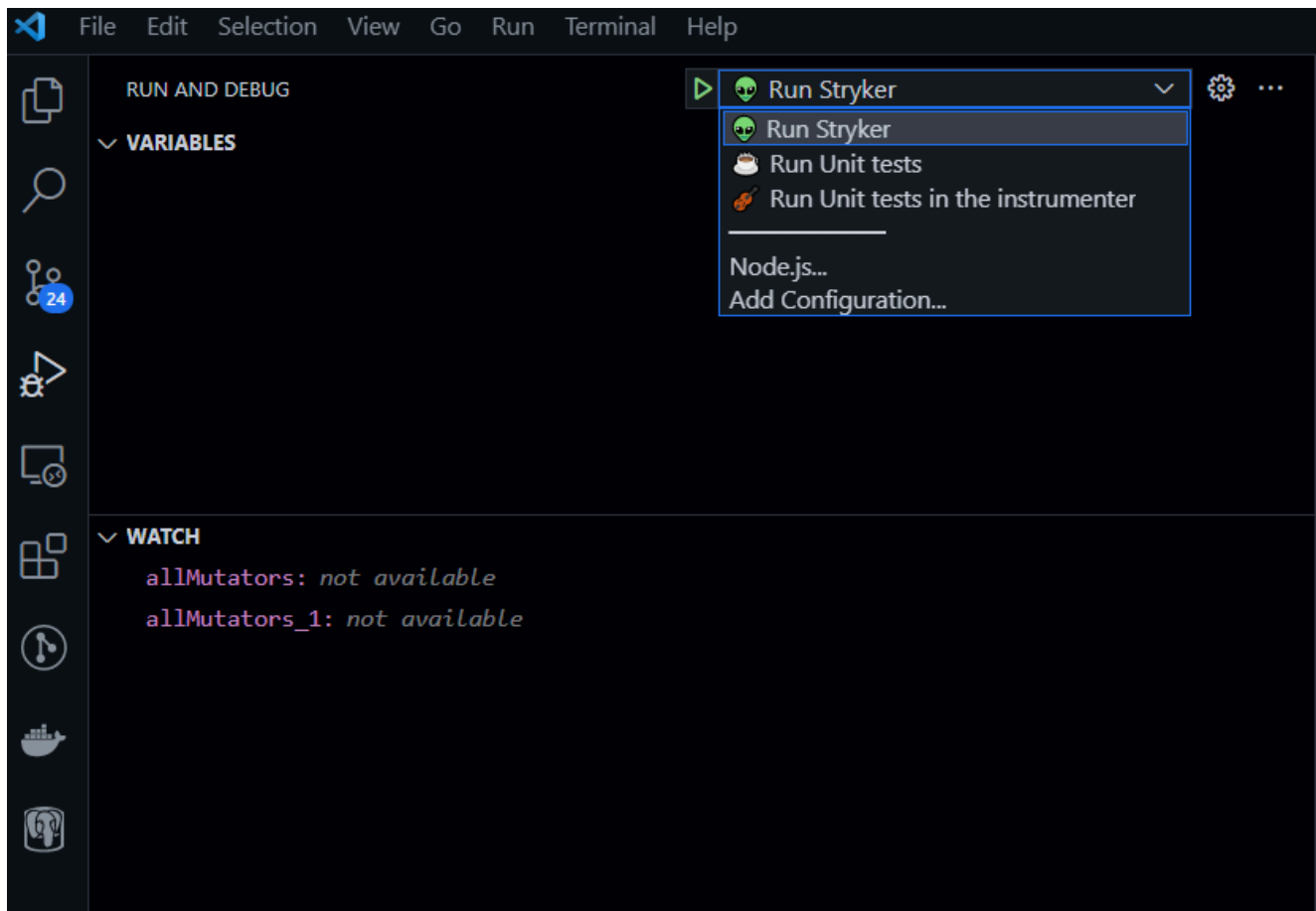
In this exercise, you will play around with numbers.js and stryker-js in order to familiarize yourself with the development environment and the code bases.

1. Open up "numbers.js/lib/numbers/basic.js". This is the JavaScript file containing the basic math operations we'll be focussing on. You can find operations like product, max and gcd here.
2. Open up "numbers.js/test/basic.test.js". This file contains the unit tests for "basic.js". You can see test declarations here, like `test('sum should return the sum of items in an array', () => {})`. You can run the unit tests using the "Run and Debug" pane and running the "☕ Run Unit tests" task

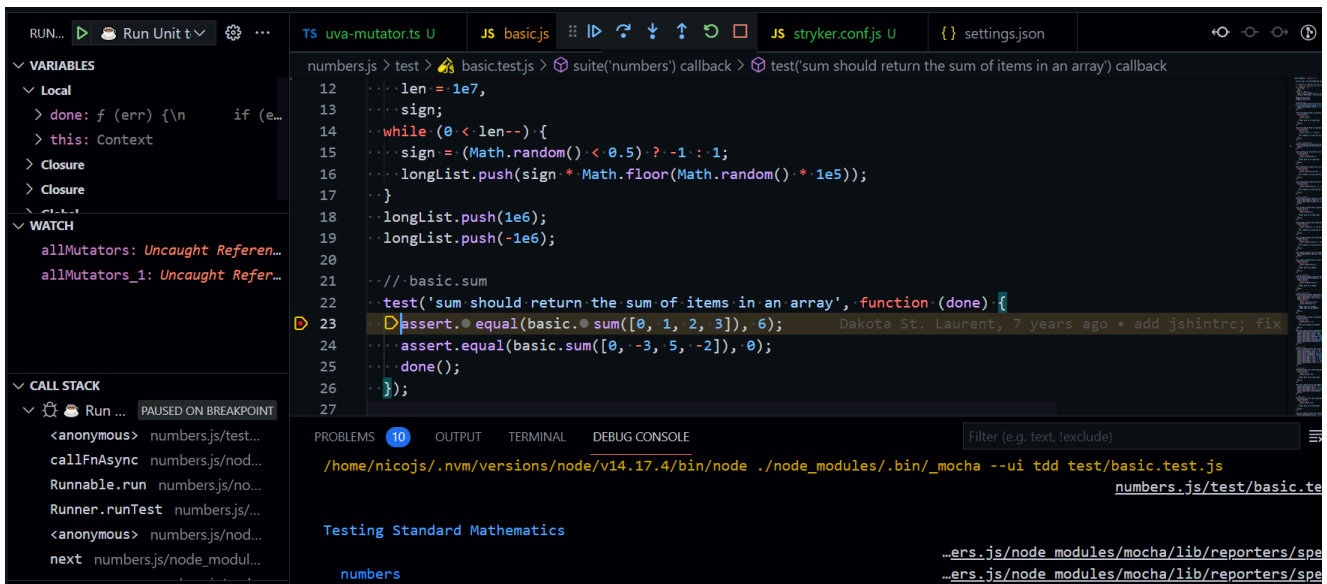
3. Open up the "stryker-js/packages/instrumenter/src/mutators" directory. Here, you'll find all mutators that StrykerJS supports out of the box. Open a few of them to familiarize yourselves with the syntax. One of the simplest is "boolean-literal-mutator.ts".
4. Notice that each mutator has a name (string) and a `*mutate(path)` (method). Each mutate implementation follows this pattern:
 1. Test to see if the AST node supports the mutation.
 2. If so, yield one or more mutations, but never change the AST node directly.
5. You can see an empty mutator called "uva-mutator.ts", this is where you can add a custom mutator yourselves.

Further more, there are 3 run tasks prepared:

- 🧐 Run Stryker
- ☕ Run Unit tests
- 🎻 Run Unit tests in the instrumenter.



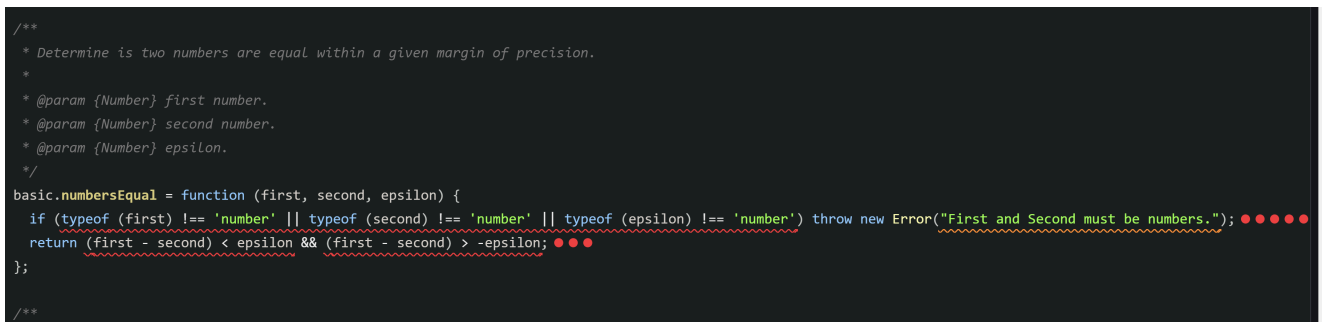
These tasks can help you by allowing you to debug the code. For example, if you want to debug a unit tests in math.js, you can debug "☕ Run Unit tests" while activating a break point in your code.



If you want to debug a mutator, you can add a breakpoint in the TypeScript file of the mutator and debug "🐞 Run Stryker".

1.3. Improve mutation score

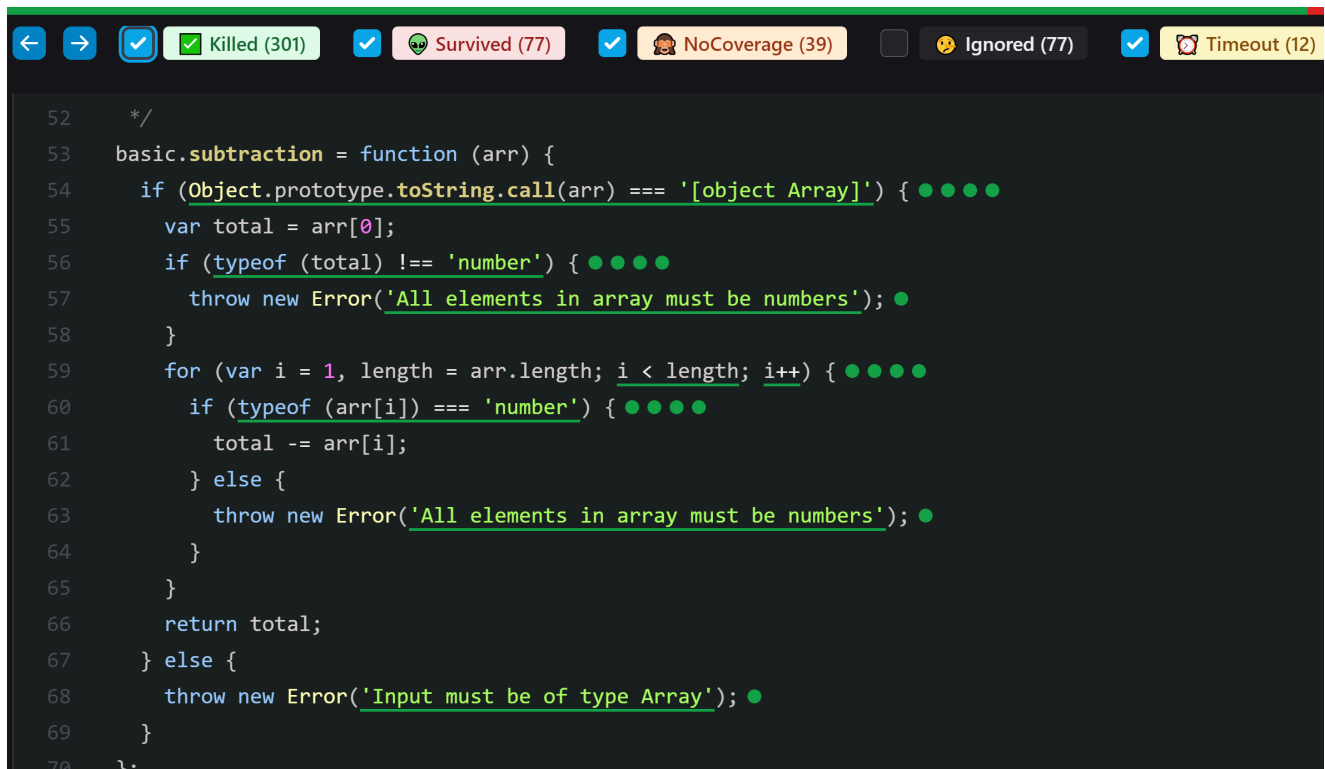
Improve mutation score of the `numbersEqual` function. The tests should pass when no mutants are active, but should kill the mutant(s) in this function.



1.4. Design a custom mutator

When you zoom in on the subtraction function, you can see that line 61 has no mutant. In fact, no mutant operator was active on this line. Is this line well tested?

Design a mutator that proves that this line of code isn't well tested. Test out your mutator by manually inserting the mutant into the code yourself and running the tests. Do the tests still pass?



The screenshot shows the StrykerJS interface with a toolbar at the top indicating: Killed (301), Survived (77), NoCoverage (39), Ignored (77), and Timeout (12). The code editor displays the following JavaScript code for `basic.subtraction`:

```

52  */
53  basic.subtraction = function (arr) {
54    if (Object.prototype.toString.call(arr) === '[object Array]') { ●●●●
55      var total = arr[0];
56      if (typeof (total) !== 'number') { ●●●●
57        throw new Error('All elements in array must be numbers'); ●
58      }
59      for (var i = 1, length = arr.length; i < length; i++) { ●●●●
60        if (typeof (arr[i]) === 'number') { ●●●●
61          total -= arr[i];
62        } else {
63          throw new Error('All elements in array must be numbers'); ●
64        }
65      }
66      return total;
67    } else {
68      throw new Error('Input must be of type Array'); ●
69    }
70  }
  
```

1.5. Implement the mutator

Implement the mutator designed in exercise 4 mutator. A empty mutator is prepared for just this occasion: "uva-mutator.ts". Make sure TypeScript compiler is running in watch mode.

Test that your mutator works by executing StrykerJS on the basic.js file. There should be a surviving mutant on line 61.

Now that we see the mutant survives, it is time to kill it. Add a test or change one of the existing tests.



2. Report

Write a short report in which you reflect on your experience with mutation testing. The following questions can get you started.

- How does the mutation testing relate to unit testing and test unit test coverage? What are the strengths and weaknesses of each?
- Summarize the promised benefits of mutation testing. Based on your experiences in this laboratory, do they hold up in practice?
- What disadvantages and limitations do you see in mutation testing? Do you see threats?
- How does mutation testing complement other testing approaches? In what situations would you apply mutation testing and when would you not use mutation testing?

You can include or append your final mutation report (html file).